Onfinity AI Orchestration Framework

Solution Architecture & Implementation Blueprint

---

1. Purpose of This Document

This document provides the foundational architecture, governance model, implementation structure, and orchestration blueprint for building scalable AI-driven ERP implementation and operational workflows within the Onfinity platform.

The objective is to establish:

- A standardized orchestration architecture
- Controlled multi-agent execution
- Governed tool usage
- Safe ERP interaction patterns
- Reusable implementation templates
- Scalable onboarding of future domain agents
- Enterprise-grade implementation governance

This document acts as:

- Solution Architecture
- Implementation Guide
- Governance Blueprint
- Runtime Design Reference
- Future Expansion Framework

---

2. High-Level Vision

The Onfinity AI Orchestration Framework is not merely a chatbot or AI assistant layer.

It is an:

Enterprise Agentic ERP Execution Platform

that enables:

- Guided ERP implementation
- Controlled ERP execution

- AI-governed business workflows
- Multi-agent orchestration
- Runtime delegation
- Validation-driven automation
- Approval-governed mutations
- Scalable operational intelligence

The architecture combines:

- AI Agents
- Tool Library
- VAAPI Services
- Runtime Delegation
- Knowledge Bases
- LLM Providers
- Governance Controls
- ERP Metadata

into a unified orchestration framework.

---

3. Core Architectural Principles

3.1 Separation of Concerns

The framework separates:

Concern	Responsibility
Orchestration	Governing execution flow
Agent	Business reasoning
Tool	System capability
Delegation	Workflow routing
Knowledge Base	Domain grounding
LLM Provider	AI execution engine
Validation	Monitoring & governance
Approval	Human checkpoint control

This separation is essential for scalability and governance.

3.2 Controlled ERP Mutation

ERP systems are highly sensitive operational systems.

Therefore:

- Read operations should be unrestricted where safe
- Validation operations should be isolated
- Write operations should be governed
- Posting operations should require approvals
- Destructive operations should be highly restricted
- Runtime execution should remain auditable

3.3 Guided Implementation Model

The framework follows a guided implementation methodology.

Typical stages:

1. Discovery
2. Validation
3. Recommendation
4. Approval Request
5. Controlled Execution
6. Verification
7. Monitoring
8. Handover

3.4 Agent Specialization

Each agent should have:

- A focused responsibility
- Defined capabilities
- Controlled tools
- Limited execution scope
- Clear delegation boundaries

Avoid monolithic “do everything” agents.

---

4. Runtime Architecture Overview

4.1 Runtime Hierarchy

```
AI Orchestration
    ↓
Supervisor Agent
    ↓
Delegation Rules
    ↓
Specialized Agents
    ↓
Tool Bindings
    ↓
VAAPI Services
    ↓
ERP Execution
```

---

5. AI Orchestration Window Structure

5.1 Window Tabs

The AI Orchestration Setup window contains:

Tab	Purpose
AI Orchestration	Root orchestration definition
AI Agent	Runtime agent definitions
Agent Tool	Tool-to-agent bindings
Agent Delegation	Runtime workflow delegation

---

6. Backend Meta Model

6.1 AI Orchestration

Table

`VAI01_AIOrchestration`

Purpose

Represents:

- Business workflow orchestration
- Implementation program
- Multi-agent execution flow
- Runtime governance context

Key Responsibilities

- Defines orchestration identity
- Maintains orchestration scope
- Links supervisor agent
- Governs overall execution

Important Fields

Field	Purpose
Search Key	Unique orchestration identifier
Task	Business objective
Description	Orchestration summary
VAI01_OAGENT_ID	Supervisor agent reference
IsActive	Runtime enablement

6.2 AI Agent

Table

`VAI01_OAgent`

Purpose

Represents runtime AI execution units.

Responsibilities

- Domain reasoning
- Workflow execution
- Tool orchestration
- Validation
- Monitoring
- Delegation

Important Fields

Field	Purpose
Name	Agent identity
VAI01_AGENTTYPE	Behavioral classification
VAI01_PROMPT	Runtime prompt
VAI01_AGENTKNOWLEDGEBASE_ID	Knowledge source
VAI01_LLMCONFIGURATION_ID	LLM binding
VAI01_LLMCUSTOMIZEPARAMETER	LLM tuning
VAI01_OTHERCUSTOMIZEPARAMETER	Runtime tuning

6.3 Agent Tool Mapping

Table

`VAI01_OAgentAPI`

Purpose

Defines the runtime capability bindings between agents and Tool Library methods.

Runtime Binding

```
Agent ↔ Tool ↔ Method
```

Important Fields

Field	Purpose
VA101_API_ID	Tool/Service

Field	Purpose
VA101_APIMETHOD_ID	Executable method
VAI01_EXECUTIONORDER	Ordered execution
VAI01_RESPONSEHANDLING	Response interpretation
VAI01_CUSTOMPARAMS	Runtime parameter injection

6.4 Agent Delegation

Table

`VAI01_OAgentDelegation`

Purpose

Defines runtime workflow routing and orchestration collaboration.

Responsibilities

- Agent handoff
- Escalation
- Workflow routing
- Delegation contracts
- Runtime coordination

Important Fields

Field	Purpose
VAI01_DELEGATETO	Target agent
VAI01_RULE	Delegation instruction/contract

---

7. Tool Library Architecture

7.0 Metadata-Driven Tool Architecture

The Tool Library is implemented as a metadata-driven capability abstraction layer.

This layer separates:

Concern	Responsibility
API Service	Capability domain
Method	Executable action
Parameter	Runtime input definition
Agent Mapping	Capability assignment
Delegation	Runtime workflow routing

The architecture enables:

- Reusable capability registration
- Agent-independent tool abstraction
- Runtime orchestration flexibility
- Import-driven onboarding
- SQL-driven provisioning
- Future automation generation

The Tool Library architecture is strongly aligned with enterprise integration platform design principles.

7.0.1 Confirmed Runtime Tool Hierarchy

```
Tool/Service
    ↓
Methods
    ↓
Parameters
```

And at runtime:

```
AI Agent
    ↓
Agent Tool Mapping
    ↓
Tool Service + Method
    ↓
VAAPI Execution
```

7.0.2 Bulk Import Architecture

The Tool Library import structure uses a flattened hierarchical import model.

Meaning:

- APIs repeat across rows
- Methods repeat across rows
- Each parameter creates an additional row
- Hierarchical runtime structure is transformed into tabular import structure

This structure is implementation-friendly for:

- Excel imports
- CSV imports
- SQL generation
- Metadata-driven automation
- Tool onboarding acceleration

7.0.3 Metadata Sources Supporting Tool Library

The following metadata layers are now identified:

Metadata File	Purpose
APILibrary_WindowStructure_V1.xml	Window/tab hierarchy
APILIbraryServices_TableColumn_V1.xml	DB column metadata
APILIbraryServices_LOV_V1.xml	Reference/list values
APILIbraryServices_Process_V1.xml	Runtime process/actions
WindowStructure_V*.xml	ERP window metadata
TableColumn_V*.xml	ERP table metadata
Process_V*.xml	ERP process metadata

These metadata layers collectively establish:

- Runtime orchestration structure
- Tool metadata abstraction
- Validation metadata
- LOV decoding
- Process integration
- Metadata-driven UI behavior

7.0.4 Confirmed Tool Library Tables

Tab	Table
Tool/Service	VA101_API
Method	VA101_APIMethod
Method Parameter	VA101_MethodParameter

7.1 Tool Library Structure

The Tool Library consists of:

Tab	Table
Tool/Service	VA101_API
Method	VA101_APIMethod
Method Parameter	VA101_MethodParameter

7.2 Runtime Tool Hierarchy

```
Tool/Service
    ↓
Method
    ↓
Parameters
```

7.3 Tool Responsibilities

VA101_API

Defines:

- Service grouping
- Base URL
- Authentication type
- Protocol
- Service ownership

VA101_APIMethod

Defines:

- Executable method
- Endpoint path
- Request format
- Response format
- Method behavior

VA101_MethodParameter

Defines:

- Parameter structure
- Data type
- Location
- Required behavior

---

8. Recommended Agent Types

8.0 Confirmed Existing Runtime Agent Types

The current orchestration runtime already supports the following agent classifications:

Existing Agent Type	Runtime Purpose
Supervisor	Governs orchestration and delegation
Action	Executes business operations
Monitoring	Validates, observes, and verifies runtime state

Delegation is currently restricted to Supervisor agents, which creates a strong centralized orchestration model.

Runtime pattern:

```
Supervisor Agent
      ↓ delegates to
Action / Monitoring Agents
```

This architecture supports:

- Controlled workflow execution
- Governance-driven orchestration
- Deterministic routing
- Centralized runtime coordination
- Safe ERP interaction

8.0.1 Recommended Future Agent Types

Agent Type	Purpose
Supervisor	Governs orchestration
Action	Executes operations
Monitoring	Validates/observes
Planning	Discovery & blueprint
Validation	Readiness verification
Approval	Human checkpoint control
Recovery	Rollback/remediation
Audit	Compliance validation
Advisory	Recommendation-only

---

9. Finance Module Orchestration Blueprint

9.1 Recommended Finance Orchestration Structure

```
Finance Supervisor Agent
    ├── Discovery & Blueprint Agent
    ├── Tenant & Organization Agent
    ├── Role & Access Control Agent
    ├── General Ledger Agent
    ├── Accounts Payable Agent
    ├── Accounts Receivable Agent
    ├── Tax Configuration Agent
    ├── Banking Agent
    ├── Asset Management Agent
    ├── Posting Validation Agent
```

```
    ├── Period Close Agent
    ├── Reconciliation Agent
    ├── Reporting Agent
    ├── Migration Agent
    ├── Go-Live Validation Agent
    └── Audit & Compliance Agent
```

---

10. Governance Framework

10.0 Runtime Governance Observations

The runtime architecture already contains multiple governance-oriented constructs:

Governance Capability	Implementation
Ordered execution	VAI01_EXECUTIONORDER
Response interpretation	VAI01_RESPONSEHANDLING
Runtime parameter injection	VAI01_CUSTOMPARAMS
Delegation contracts	VAI01_RULE
Tool isolation	Agent Tool mapping
Controlled routing	Supervisor delegation

These runtime constructs establish a strong governance baseline for enterprise ERP execution.

10.0.1 Execution Sequencing

The presence of:

```
VAI01_EXECUTIONORDER
```

confirms support for deterministic orchestration pipelines.

Example Finance flow:

1. Validate tenant
2. Validate organization
3. Validate accounting schema

4. Create calendar
5. Create periods
6. Configure dimensions
7. Validate posting setup
8. Execute posting validation
9. Generate readiness report

10.0.2 Response Handling

The presence of:

```
VAI01_RESPONSEHANDLING
```

indicates support for:

- Runtime interpretation
- Conditional orchestration
- Delegation triggers
- Validation analysis
- Structured execution outcomes

This becomes highly valuable for Finance orchestration and validation agents.

10.0.3 Delegation Contracts

The field:

```
VAI01_RULE
```

acts as a delegation contract layer.

This allows:

- Contextual routing guidance
- Input expectations
- Workflow instructions
- Runtime orchestration semantics

Example:

```
Use this agent to validate accounting schema readiness before GL setup
execution.
```

10.0.4 Agent Runtime Manifest Model

Each AI Agent effectively behaves as a self-contained runtime manifest containing:

Capability	Source
Reasoning	Prompt
Knowledge	Knowledge Base
Runtime Model	LLM Configuration
Capability Access	Tool Bindings
Workflow Routing	Delegation Rules
Runtime Tuning	Custom Parameters

This is an enterprise-grade agent runtime abstraction model.

10.1 Execution Classification

Recommended future classification:

Classification	Purpose
Read	Data retrieval
Validate	Verification
Simulate	Dry-run planning
Execute	Controlled write
Post	Accounting mutation
Admin	System-level operations
Destructive	Delete/reset operations

10.2 Approval Requirements

Recommended policy:

Operation Type	Approval Requirement
Read	No approval
Validation	No approval
Simulation	No approval
Configuration Write	Approval recommended
Posting	Mandatory approval
Period Close	Mandatory approval
Delete/Reset	Restricted approval

---

11. Delegation Framework

11.1 Delegation Philosophy

Only Supervisor agents should orchestrate delegation.

Delegation rules should:

- Define handoff intent
- Describe required inputs
- Define expected outputs
- Clarify execution purpose
- Maintain governance boundaries

11.2 Delegation Pattern

```
Supervisor Agent
      ↓
Delegates Task
      ↓
Action Agent
      ↓
Returns Response
```

```
      ↓
Supervisor Validates Outcome
```

---

12. Runtime Execution Model

12.1 Execution Lifecycle

Typical runtime execution:

1. Orchestration activated
2. Supervisor agent invoked
3. Delegation rules evaluated
4. Specialized agent selected
5. Tool bindings resolved
6. Tool execution performed
7. Response interpreted
8. Validation executed
9. Approval requested if needed
10. Final execution completed
11. Monitoring performed
12. Audit trail persisted

---

13. Knowledge Base Strategy

13.1 Knowledge Separation

Recommended separation:

Knowledge Type	Purpose
Central Knowledge Base	Shared organizational knowledge
Agent Knowledge Base	Domain specialization
Tool Metadata	Runtime capability guidance
Governance Knowledge	Compliance and controls
ERP Metadata	System structure/context

---

14. Recommended Future Enhancements

14.1 Tool Classification Enhancements

Recommended future metadata:

Suggested Field
Risk Level
Requires Approval
Read/Write Classification
Retry Allowed
Rollback Supported
Finance Criticality
Posting Impact
Delegation Allowed

14.2 Delegation Enhancements

Recommended future capabilities:

- Conditional delegation
- Escalation chains
- Retry orchestration
- Parallel execution
- Dynamic routing
- Runtime validation checkpoints

---

15. Strategic Conclusion

The architecture has now evolved beyond simple AI assistants.

The Onfinity platform is effectively becoming:

supporting:

- Multi-agent orchestration
- Metadata-driven runtime behavior
- AI-governed ERP execution
- Runtime delegation
- Tool capability abstraction
- Guided ERP implementation
- Controlled ERP mutation
- Governance-driven execution
- Reusable implementation frameworks
- Scalable onboarding of future AI agents

15.1 Current Architectural Maturity

Area	Status
Orchestration model	Strong
Agent runtime model	Strong
Delegation framework	Strong
Tool abstraction	Strong
Metadata visibility	Strong
Guided implementation pattern	Strong
Governance model	Strong
SQL automation readiness	Ready
Finance orchestration vision	Strong
Runtime scaling readiness	Strong

15.2 Strategic Platform Understanding

The architecture now clearly operates as:

```
AI Orchestration Layer
        ↓
Agent Runtime Layer
        ↓
Tool Capability Layer
        ↓
VAAPI Execution Layer
        ↓
ERP Runtime Layer
```

This separation creates a scalable enterprise-grade orchestration platform.

15.3 Finance Implementation Vision

The Finance orchestration initiative is no longer foundational experimentation.

The platform foundation is now sufficiently mature for:

- Standardized orchestration templates
- Reusable finance agent archetypes
- Controlled implementation workflows
- Guided ERP deployment
- Runtime validation pipelines
- Approval-driven execution
- Metadata-driven onboarding
- SQL automation generation

The remaining effort is primarily:

- Standardization
- Industrialization
- Governance refinement
- Tool normalization
- Finance-domain scaling
- Runtime optimization

15.4 Strategic Direction

The long-term platform direction is:

AI-Governed ERP Implementation & Operations Platform

capable of supporting:

- ERP implementation
- ERP operations
- Governance workflows
- Validation automation
- Compliance orchestration
- Monitoring agents
- Audit agents
- Recovery orchestration
- Operational intelligence

The architecture is coherent, scalable, metadata-driven, and implementation-ready.

The Onfinity AI Orchestration Framework already contains the foundational architecture required for building:

An Enterprise AI-Governed ERP Implementation & Operations Platform

The platform architecture now supports:

- Multi-agent orchestration
- Runtime capability abstraction
- Delegated workflow execution
- Controlled ERP interaction
- Guided implementation methodology
- Governance-driven execution
- Scalable onboarding of future agents
- Enterprise operational safety

The remaining effort is primarily:

- Standardization
- Expansion
- Tool industrialization
- Governance refinement
- Runtime optimization
- Finance-domain implementation scaling

The architecture is coherent, scalable, and implementation-ready.

---

16. Recommended Immediate Next Steps

Phase 1

- Finalize naming standards
- Define tool classification taxonomy
- Define orchestration standards
- Define execution governance standards

Phase 2

- Build reusable SQL templates
- Normalize Tool Library entries
- Standardize agent prompts
- Create reusable orchestration patterns

Phase 3

- Implement Finance Supervisor orchestration
- Onboard Finance domain agents
- Create validation agents
- Create monitoring agents

Phase 4

- Add audit orchestration
- Add recovery orchestration
- Add approval workflows
- Add operational dashboards

End of Document